MERRIMACK COLLEGE

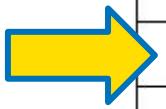
CSC 6033 Week 5

Pushdown Automata

Languages, Automata, and Decidability - Dr. Paulo Fernandes

Timeline

Where are we?



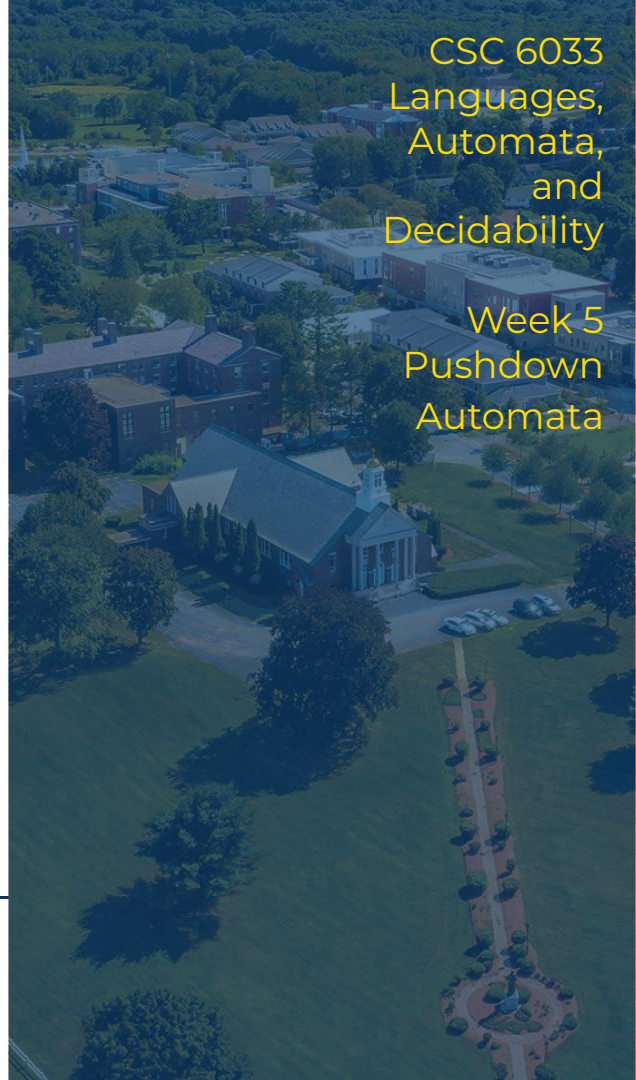
Unit	Week	Topic	Projects	Quizzes	In-class Exercises	Exam
1	1	From Python to C++	P#1	Q#1	E#1	
1	2	Developing with C++	P#2	Q#2	E#2	
2	3	Finite Automata	P#3	Q#3	E#3	
2	4	Regular Expressions	P#4	Q#4	E#4	
3	5	Pushdown Automata	P#5	Q#5	E#5	
4	6	Turing Machines	P#6	Q#6	E#6	
5	7	Processing Input and Output	P#7	Q#7	E#7	
6	8	Decidability				Final Exam (all units)

Don't let tasks pile up!

Presentation Agenda

Week 5

- Context-Free Grammars
  a. Languages from Rules
  b. Productions-based grammars
  c. Regular and Context-Free Languages
- Pushdown Automata
  a. PDA Finite State Machine
  b. PDA and CFG
  c. Matching characters example
- This Week's tasks



Context-Free Grammars

"Grammar is the logic of speech, even as logic is the grammar of reason."

Richard Chenevix Trench

A grammar is an alternative way to describe a language. As such, a grammar is the way to describe through rules the way words of the language are formed.

When we formally define this rules into "productions" we formally define a context-free languages.

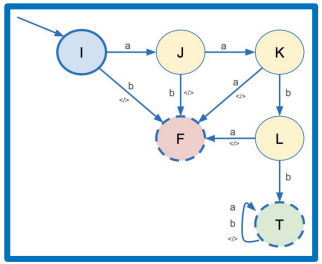
- **Languages from Rules**
- Productions-based Grammars
- Regular and Context-Free Languages

Previously in Lang., Aut. and Decid.

We have seen Regular languages being defined as FSA (DFA and NFA) and RegEx.

However, while transforming a DFA into a RegEx we define generically the language associated to each state of an DFA as a composition of the previous origins and the letters of the incoming arcs.

Such a way to define those languages associated to each state (ultimately to define the language for the final states) can be seen as the establishment of rules to generate/accept words of the language.



I = **ε**

J = **Ia** becomes **J** = **εa** = **a**

K = **Ja** becomes **K** = **aa**

L = **Kb** becomes **L** = **aab**

T = **Lb** + **Ta** + **Tb**

aabb(a|b)*

MERRIMACK COLLEGE

What if we define directly a language from rules?

Languages from rules - Example 1

It defines the ***{a, b}*** alphabet.

For example, let's define a language by the following rules:

- §1. The word ***aa*** can be accept by the language;
- §2. If a word **W** can be accepted in the language, the word **W*a*** also can;
- §3. If a word **W** can be accepted in the language, the word **W*b*** also can.

For instance, this will produce the words:

- ***aa***, because of §1;
- ***aaa*** and ***aab***, because of ***aa*** then §2 and ***ε*** then §3;
- ***aaaa*** and ***aaab***, because of ***aaa*** then §2 and ***aaa*** then §3;
- ***aaba*** and ***aabb***, because of ***aab*** then §2 and ***aab*** then §3;
- ***aaaaa*** and ***aaaab***, because of ***aaaa*** then §2 and ***aaaa*** then §3;
- ***aaaba*** and ***aaabb***, because of ***aaab*** then §2 and ***aaab*** then §3;

The same goes for:

- ***aabaa,***
  aabab,
  aabba,
  aabbb,
  ...

Conversely, this language has all words starting with ***aa*** and followed by any others letters.

Languages from rules - Example 2

It defines the ***{a, b}*** alphabet.

For example, let's define a language by the following rules:

- §1. The empty word (***ε***) can be accepted by the language;
- §2. If a word **W** can be accepted by the language, the word ***a*W** also can;
- §3. If a word **W** can be accepted by the language, the word **W*b*** also can.

For instance, this will produce the words:

- ***ε***, because of §1;
- ***a*** and ***b***, because of ***ε*** then §2 and ***ε*** then §3;
- ***aa*** and ***ab***, because of ***a*** then §2 and ***a*** then §3;
- ***ab*** and **bb**, because of ***b*** then §2 and ***b*** then §3;

The same goes for:

- ***aaa, aab, abb, bbb, aaaa, aaab, aabb, abbb, bbbb, aaaaa, aaaab, ...***

Conversely, this language has ***ε*** and all words with ***a***'s packaged on the left, ***b***'s packaged on the right.

Languages from rules - Example 3

It defines the ***{a, b}*** alphabet.

Another example of language definition by rules can be:

- §1. The empty word (***ε***) can be accepted by the language;
- §2. If a word **W** can be accepted by the language, the word ***aWa*** also can;
- §3. If a word **W** can be accepted by the language, the word ***bWb*** also can.

For instance, this will produce the words:

- ***ε***, because of §1;
- ***aa*** and ***bb***, because of ***ε*** then §2 and ***ε*** then §3;
- ***aaaa*** and ***baab***, because of ***aa*** then §2 and ***aa*** then §3;
- ***abba*** and ***bbbb***, because of ***bb*** then §2 and ***bb*** then §3;

Note: A palindrome is a word that is equal to its reverse, as ***I***, ***OO***, ***mom***, ***deed***, ***level***, ***redder***, and ***rotator*** (for English).

The same goes for:

- ***aaaaaa,***
  baaaab,
  abaaba,
  bbaabb,
  aabbaa,
  babbab, ...

Conversely, this language is called even-letters-palindrome.

Languages from rules - Example 4

It defines the ***{a, b}*** alphabet.

Another example of language definition by rules can be:

- §1. The word ***a*** can be accept by the language;
- §2. The word ***b*** can be accept by the language;
- §3. If a word **W** can be accepted in the language, the word ***a*W*a*** also can;
- §4. If a word **W** can be accepted in the language, the word ***b*W*b*** also can.

For instance, this will produce the words:

- ***a*** and ***b***, because of §1 and §2;
- ***aaa*** and ***bab***, because of ***a*** then §3 and ***a*** then §4;
- ***aba*** and ***bbb***, because of ***a*** then §3 and ***a*** then §4;
- ***aaaaa*** and ***baaab***, because of ***aaa*** then §3 and ***a*** then §4;
- ***ababa*** and ***bbabb***, because of ***bab*** then §3 and ***a*** then §4;

The same goes for:

- ***aabaa, babab, abbba, bbbbb, aaaaaaa, baaaaab, aababaa, ...***

Conversely, this language is called odd-letters-palindrome.

Context-Free Grammars

"Grammar is the logic of speech, even as logic is the grammar of reason."

Richard Chenevix Trench

A grammar is an alternative way to describe a language. As such, a grammar is the way to describe through rules the way words of the language are formed.

When we formally define this rules into "productions" we formally define a context-free languages.

- Languages from Rules
- **Productions-based Grammars**
- Regular and Context-Free Languages

Productions-based Grammars

Given a set of rules, a language ***L*** can be defined by a **grammar** that defines:

- an alphabet (***Σ***) composed by all acceptable symbols (including letters and strings) that we will call **terminals**;
- a set of identifiers (***NT***) that we call **non-terminals**, in which one of them is considered the initial one and is named after the name of the language (***L***) the full definition of the language starts from this non-terminal identifier;
- a finite set of productions of the form:
  - one **non-terminal** ⇒ finite string of **non-terminals** and/or **terminals**.

Those finite strings can be composed by non-terminals alone, terminals alone, or any mixture of both. To avoid confusion we will denote non-terminals as capital letters identifiers, and terminals as lowercase letter identifiers or symbols.

The grammars defined as such are called **Context-Free Grammars** - **CFG**.

Context-Free Grammars - CFG - Example 1

A CFG is said to be context-free because the rule expressed by a production can be applied regardless of the productions applied before or after.

For example, let's revisit the language of Example 1 (words starting with ***aa***) and called it ***S***:

- ***Σ = {a, b}***
- ***NT = {S}***

Productions:

P1	***S ⇒ Sa***
P2	***S ⇒ Sb***
P3	***S ⇒ aa***

For instance, ***aaab*** is produced by:

P2 : ***Sb***	***<S>b***
P1: ***Sa***	***<S>ab***
P3: ***aa***	***aaab***

- §1. The word ***aa*** can be accept by the language;
- §2. If a word **W** can be accepted in the language, the word **Wa** also can;
- §3. If a word **W** can be accepted in the language, the word **Wb** also can.

Text: Context-free Grammar.

Context-Free Grammars - CFG - Example 2

Let's revisit the language of Example 2 (words either empty or words with ***a***'s on the left and ***b***'s on the right), and called it ***S***:

- ***Σ = {a, b}***
- ***NT = {S}***

Productions:

P1	***S ⇒ aS***
P2	***S ⇒ Sb***
P3	***S ⇒ ε***

For instance, ***aaab*** is produced by:

P1 : ***aS*** ***a<S>***
P1: ***aS*** ***aa<S>***
P1: ***aS*** ***aaa<S>***
P2: ***Sb*** ***aaa<S>b***
P3: ***εaaaεb***

- §1. The empty word (***ε***) can be accepted by the language;
- §2. If a word **W** can be accepted by the language, the word ***a*W** also can;
- §3. If a word **W** can be accepted by the language, the word **W*b*** also can.

Note that ***ε*** inside a word represents no letter.

Context-Free Grammars - CFG - Example 3

Let's revisit the language of Example 3 (even-letters-palindrome), and called it ***S***:

- ***Σ = {a, b}***
- ***NT = {S}***

Productions:

P1	***S ⇒ aSa***
P2	***S ⇒ bSb***
P3	***S ⇒ ε***

For instance, ***abba*** is produced by:
P1 : ***aSa*** ***a<S>a***
P2: ***bSb*** ***ab<S>ba***
P3: ***εabεba***

- §1. The empty word (***ε***) can be accepted by the language;
- §2. If a word **W** can be accepted by the language, the word ***aWa*** also can;
- §3. If a word **W** can be accepted by the language, the word ***bWb*** also can.

Note that ***ε*** inside a word represents no letter.

Context-Free Grammars - CFG - Example 4

Let's revisit the language of Example 4 (odd-letters-palindrome), and called it ***S***:

- ***Σ = {a, b}***
- ***NT = {S}***

Productions:

P1	***S ⇒ aSa***
P2	***S ⇒ bSb***
P3	***S ⇒ a***
P4	***S ⇒ b***

For instance, ***ababa*** is produced by:

P1 : ***aSa***	***a<S>a***
P2: ***bSb***	***ab<S>ba***
P3: ***a***	***ababa***

- §1. The word ***a*** can be accept by the language;
- §2. The word ***b*** can be accept by the language;
- §3. If a word **W** can be accepted in the language, the word ***aWa*** also can;
- §4. If a word **W** can be accepted in the language, the word ***bWb*** also can.

Text: Context-free Grammar.

Context-Free Grammars

"Grammar is the logic of speech, even as logic is the grammar of reason."

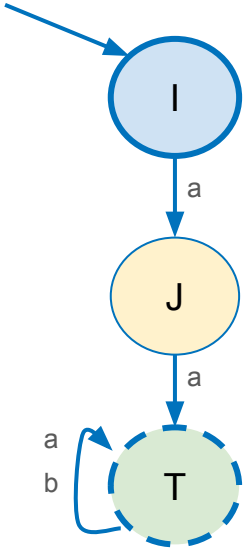
Richard Chenevix Trench

A grammar is an alternative way to describe a language. As such, a grammar is the way to describe through rules the way words of the language are formed.

When we formally define this rules into "productions" we formally define a context-free languages.

- Languages from Rules
- Productions-based Grammars
- **Regular and Context-Free Languages**

RegEx and CFG



We know from previous week topics that the language define for Example 1 is a regular language, that can very well be described as a RegEx by:

- ***aa(a|b)****

This same language can defined using CFG as we seen by:

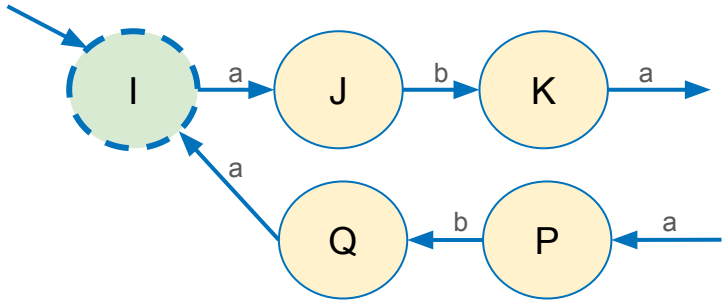
- ***Σ = {a, b}***
- ***NT = {S}***

P1 ***S ⇒ Sa***
P2 ***S ⇒ Sb***
P3 ***S ⇒ aa***

In fact, all regular languages, i.e., languages definible by a RegEx, can also be defined by a CFG, thus they are also a context-free language. The formal proof is described in the video below, but this is though one...

Video: Regular Languages are Context-Free & More!.

RegEx and CFG



However, some context-free languages cannot be represented by a RegEx. For example, this is the case of the language of Example 3 (even-letters-palindrome) that is defined using CFG:

- ***Σ = {a, b}***
- ***NT = {S}***

P1 ***S ⇒ aSa***
P2 ***S ⇒ bSb***
P3 ***S ⇒ ε***

In fact, a regular language has as memory only the current state (thinking about Finite Automata), but since the context-free language even-letters-palindrome as to take into account which letters have appeared on the left-hand side to reproduce it in the reverse order in the right-hand side of the words, it would require infinite states to keep the memory of the previous letters.

If a finite automata is not enough to describe a language, the language is not regular.

Pushdown Automata

"You don't know how far you can go until you push it."

Dawn Richard

To define an context-free language, we can define a CFG, but there is one kind of automata that is capable of keep the memory of what happen, thus able to judge the acceptance of context-free languages.

Such automata are called Pushdown Automata (PDA), as they have a memory structure that behaves as the stack data structure.

- **PDA Finite State Machine**
- PDA and CFG
- Matching characters example

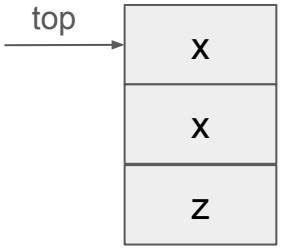
Pushdown Automata - PDA

A Pushdown Automata (PDA) is actually composed by three structures:

- An entry string (or ***tape***) holding one letter at a time followed by the end of string symbol (a constant data structure);
- A ***stack*** capable of holding letters in a stack data structure (a variable data structure) capable of be the target of POP and PUSH operations;
- The ***PDA finite state machine*** that access the tape and stack and decides if the string in the tape is to be accepted by a context-free language.

The tape, a string (list data structure).

Tape holding the word ***aabb***.



The stack, a LIFO data structure.

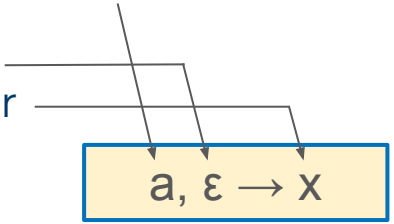
Stack holding letters (***x*** and ***z***).

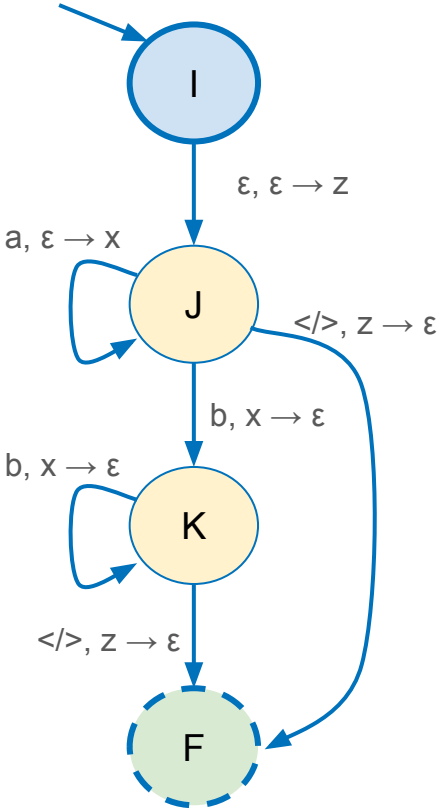
Text: Pushdown automaton.

PDA Finite State Machine

A PDA Finite State Machine can be graphically represented as a graph that is composed by vertices (states) and edges (transitions):

- The vertices are similar to FSA seem before:
  - one initial state;
  - one or more final states.
- The edges are annotated with:
  - the letter read from the tape (if any);
  - the letter resulting from a pop over the stack (if any);
  - the letter to be pushed over the stack (if any).





Video: What is a Pushdown Automaton (PDA)?.

PDA Finite State Machine

For example, the CFG for a language that holds the words with an equal number of a's preceding b's, i.e. the words:

- ***ε***, ***ab***, ***aabb***, ***aaabbb***, ***aaaabbbb***, ***...***

This language can be generically described as: $L = \{ a^n b^n \}$ where ***n*** is any number in N_0 (Naturals with zero).

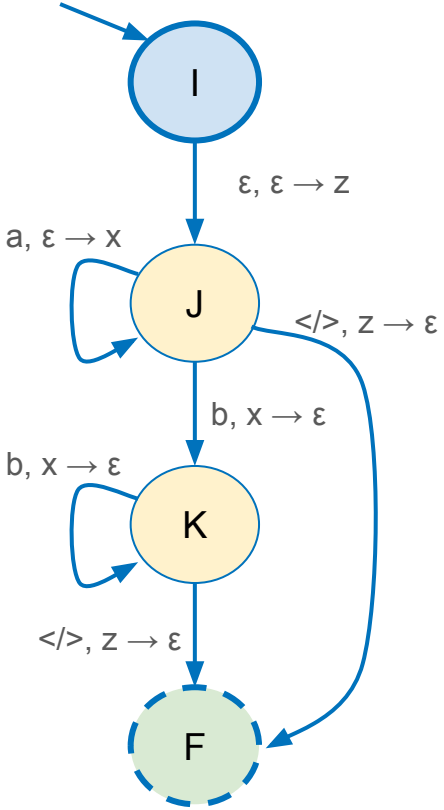
It can be simply defined by:

- ***Σ = {a, b}***
- ***NT = {S}***

P1 ***S ⇒ aSb***

P2 ***S ⇒ ε***

An be implemented by this PDA:



MERRIMACK COLLEGE

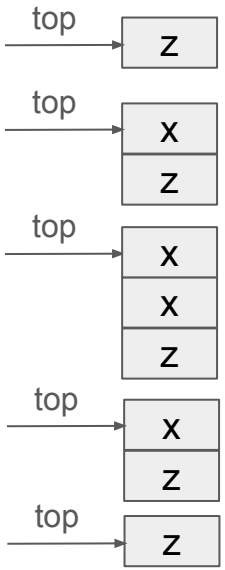
Video: Designing CFG for L = $a^n b^n$.

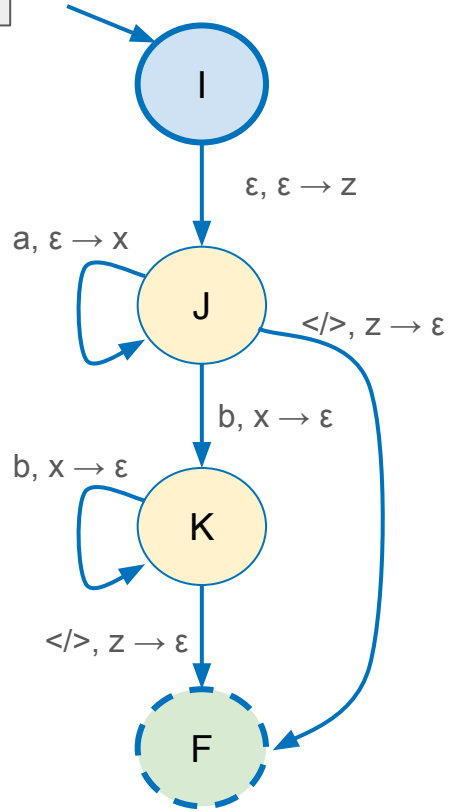
a	a	b	b	</>

PDA Finite State Machine

Now, "running" this PDA for the word ***aabb***:

1. The PDA starts on ***I***, then move to ***J*** pushing ***z*** into the stack;
2. On ***J*** an ***a*** is read from the tape, and an ***x*** is pushed on the stack;
3. On ***J*** an ***a*** is read from the tape, and an ***x*** is pushed on the stack;
4. On ***J*** a ***b*** is read from the tape, and an ***x*** is popped from the stack;
5. On ***K*** a ***b*** is read from the tape, and an ***x*** is popped from the stack;
6. On ***K*** a ***</>*** is read from the tape, and an ***z*** is popped from the stack.





MERRIMACK COLLEGE

Video: PDA example $0^n 1^n$.

Pushdown Automata

"You don't know how far you can go until you push it."

Dawn Richard

To define an context-free language, we can define a CFG, but there is one kind of automata that is capable of keep the memory of what happen, thus able to judge the acceptance of context-free languages.

Such automata are called Pushdown Automata (PDA), as they have a memory structure that behaves as the stack data structure.

- PDA Finite State Machine
- **PDA and CFG**
- Matching characters example

PDA and CFG

All context-free languages can be described both as context-free grammar (CFG) and also as a pushdown automata (PDA).

In fact, it is proven that all CFG has at least one equivalent PDA, as well as all PDA has at least one equivalent CFG.

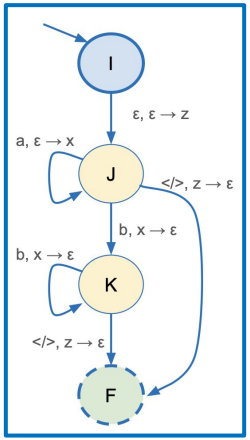
We won't see the formal equivalence, but this described in the videos links below.

Either way, CFG and PDA are equivalent formalisms, and we will see the conversion of the previously seen examples.

- $\Sigma = \{a, b\}$
- $NT = \{S\}$

P1 $S \Rightarrow aSb$

P2 $S \Rightarrow \varepsilon$



$a^n b^n$

Video: Equivalence of CFG and PDA (Part 1) - Part 2a - Part 2b.

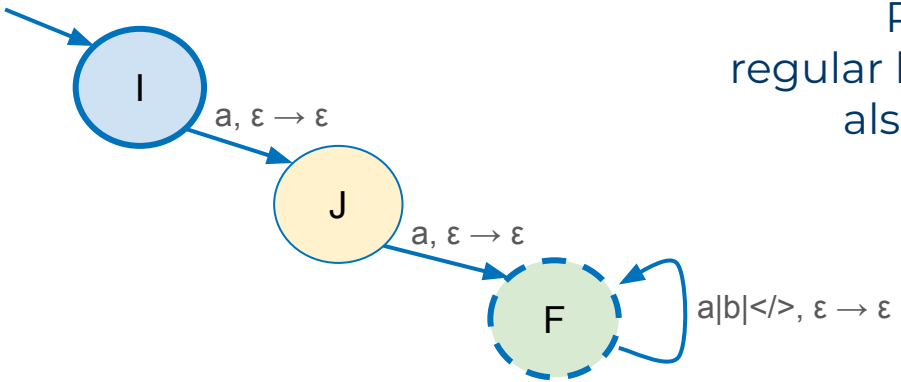
PDA and CFG - Example 1

Let's revisit the language of Example 1 (words starting with ***aa***):

- ***Σ = {a, b}***
- ***NT = {S}***

Productions:

P1	***S ⇒ Sa***
P2	***S ⇒ Sb***
P3	***S ⇒ aa***



This PDA does not perform any push or pop, and because of it it does describes a regular language.

Remember, all regular languages are also context-free languages.

Note that this PDA is equivalent to its FSA counterpart.

PDA and CFG - Example 2

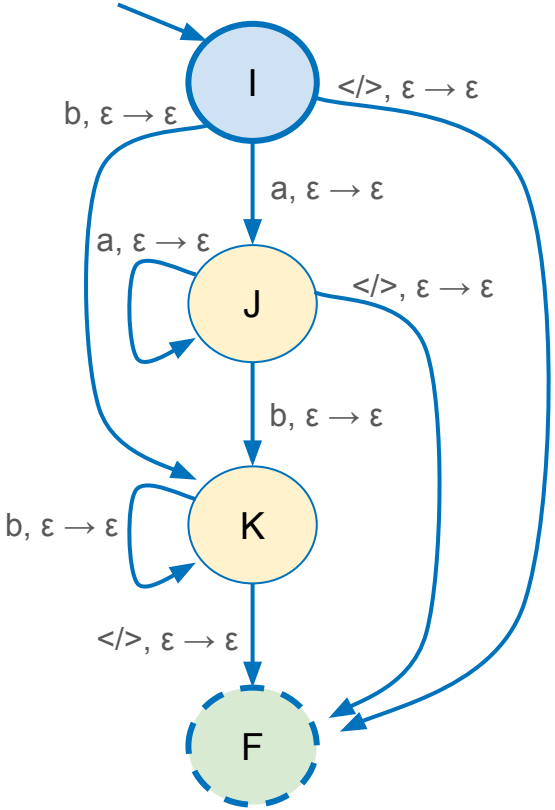
Let's revisit the language of Example 2 (words either empty or words with ***a***'s on the left and ***b***'s on the right):

- ***Σ = {a, b}***
- ***NT = {S}***

Productions:

P1	***S ⇒ aS***
P2	***S ⇒ Sb***
P3	***S ⇒ ε***

This PDA is also similar to its FSA counterpart.



Note that this PDA is also does not use the stack.

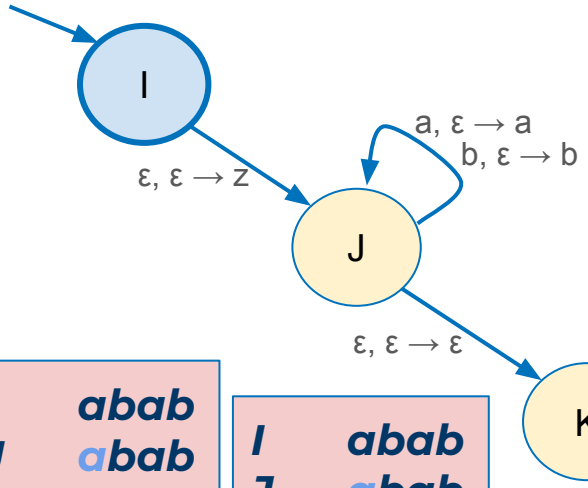
PDA and CFG - Example 3

Let's revisit the language of Example 3 (even-letters-palindrome):

- ***Σ = {a, b}***
- ***NT = {S}***

Productions:

P1	***S ⇒ aSa***
P2	***S ⇒ bSb***
P3	***S ⇒ ε***



I	***abab***
J	***abab***
J	***abab***
J	***abab***
K	***abab***

I	***abab***
J	***abab***
J	***abab***
K	***abab***

I	***abba***
J	***abba***
J	***abba***
K	***abba***
K	***abba***
F	***abba***

Note that this PDA is non deterministic. However, all even-letters-palindrome words have a valid path to be accepted, while invalid ones cannot be accepted.

Note that this PDA is non deterministic.

PDA and CFG - Example 4

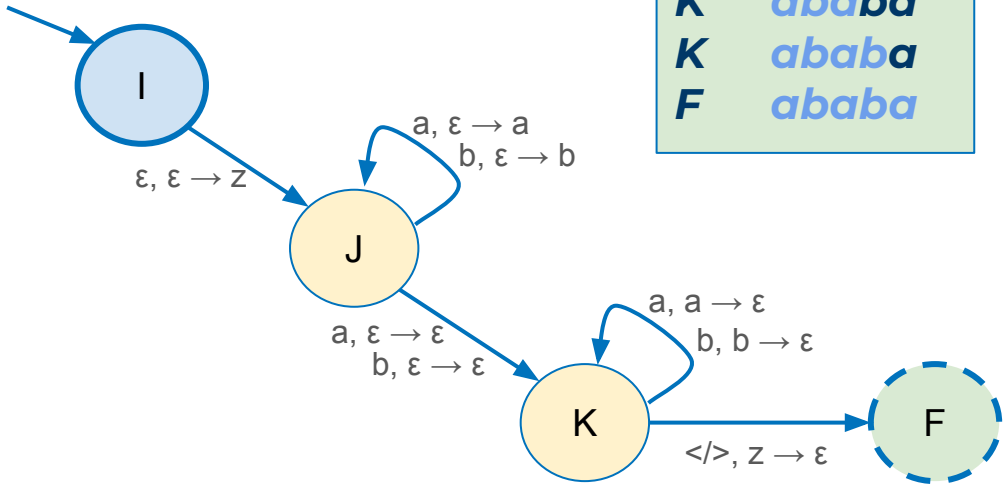
Let's revisit the language of Example 4 (odd-letters-palindrome):

- ***Σ = {a, b}***
- ***NT = {S}***

Productions:

P1	***S ⇒ aSa***
P2	***S ⇒ bSb***
P3	***S ⇒ a***
P4	***S ⇒ b***

I	***ababa***
J	***ababa***
J	***ababa***
K	***ababa***
K	***ababa***
F	***ababa***



Note that this PDA is non deterministic.

Pushdown Automata

"You don't know how far you can go until you push it."

Dawn Richard

To define an context-free language, we can define a CFG, but there is one kind of automata that is capable of keep the memory of what happen, thus able to judge the acceptance of context-free languages.

Such automata are called Pushdown Automata (PDA), as they have a memory structure that behaves as the stack data structure.

- PDA Finite State Machine
- PDA and CFG
- **Matching characters example**

Matching characters example

Considering the alphabet ***Σ = { a-z, ƀ, (,), [,] }*** let's define a language where there are lowercase letters, the blank space (denoted by ***ƀ***), plus the open/close parenthesis, and the open/close brackets. The language restriction is that every open parenthesis must be matched by and close parenthesis, and, in the same way, open and close brackets must match.

For example:

- ***a pig (pork [a latin word]) is nourishing*** is a valid word in the language;
- While ***a pig (pork [a latin word)] is nourishing*** is an invalid word.

CFG

- ***Σ = { a-z, ƀ, (,), [,] }***
- ***NT = {S}***

Productions:

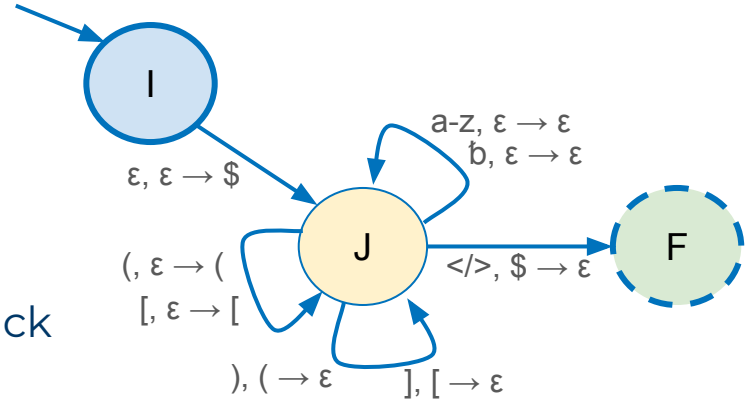
P1	***S ⇒ (S)***
P2	***S ⇒ [S]***
P3	***S ⇒ a-z|ƀS***
P4	***S ⇒ Sa-z|ƀ***
P5	***S ⇒ a-z|ƀ|ε***

This language is an example often found in practical applications.

Matching characters example

Equivalent PDA

- starts pushing in the stack a control symbol ***$***;
- each simple letter (***a-z*** or ***ƀ***) is just consumed;
- each open parenthesis or bracket is pushed into the stack;
- each close parentheses or bracket pops the corresponding open character;
- at the end the stack must be empty.



CFG

- ***Σ = { a-z, ƀ, (,), [,] }***
- ***NT = {S}***

Productions:

P1	***S ⇒ (S)***
P2	***S ⇒ [S]***
P3	***S ⇒ a-z|ƀS***
P4	***S ⇒ Sa-z|ƀ***
P5	***S ⇒ a-z|ƀ|ε***

Text: A Brutal Look at Balanced Parentheses.

This Week's tasks

- In-class Exercise E#5
- Coding Project P#5
- Quiz Q#5

Tasks

- Create the CFG and PDA to accept even and odd palindromes
- Create a C++ program to accept a string accepted by the matching characters language as previously defined
- Quiz #5 about this week topics.

In-class Exercise - E#5

Considering the Examples 3 and 4 (even-letters-palindrome and odd-letters-palindrome) presented in this class (see slides 8, 9, 14, 15, 28, and 29), you have to merge those in order to create:

- the CFG to accept palindromes with both even and odd numbers of letters;
- the PDA to accept palindromes with both even and odd numbers of letters;

Both CFG and PDA should be saved into a single ***.pdf*** file. Feel free to add remarks about your adaptation process.

You have to submit the ***.pdf*** file with your work on Canvas within the deadline.

This task counts towards the In-class Exercises grade and the deadline is This Saturday.

Fifth Coding Project - P#5

Create a C++ program to accept a string accepted by the matching characters language as previously defined (see slides 31 and 32).

Your code should:

- receive a string with the word to be analyzed. This must include only the letters of the defined alphabet (***Σ = { a-z, b, (,), [,] }***).
- create and manage a stack data structure using whatever implementation you want (even a limited sized array is acceptable);
- decide upon the end of the string, if the word is accepted or rejected.

Create also a **main** function that receives a string to be analyzed and delivers the result computed by the PDA.
Your task: Go to Canvas, and submit your C++ file (.cpp) within the deadline.

This assignment counts towards the Projects grade and the deadline is Next Tuesday.

Fifth Quiz - Q#5

- The fifth quiz in this course covers the topics of Week 5;
- The quiz will be available this Saturday, and it is composed by 10 questions;
- The quiz should be taken on Canvas (Module 5), and it is not a timed quiz:
  - You can take as long as you want to answer it;
- The quiz is open book, open notes, and you can even use any language Interpreter to answer it;
- Yet, the quiz is evaluated and you are allowed to submit it only once.

Your task:

- Go to Canvas, answer the quiz and submit it within the deadline.

This quiz counts towards the Quizzes grade and the deadline is Next Tuesday.

Week 5 of CSC 6033

- Do In-class Exercise E#5 until Saturday;
- Do Quiz Q#5 (available Saturday) until next Tuesday;
- Do Coding Project P#5 until next Tuesday.

Next Week - Turing Machines

MERRIMACK COLLEGE